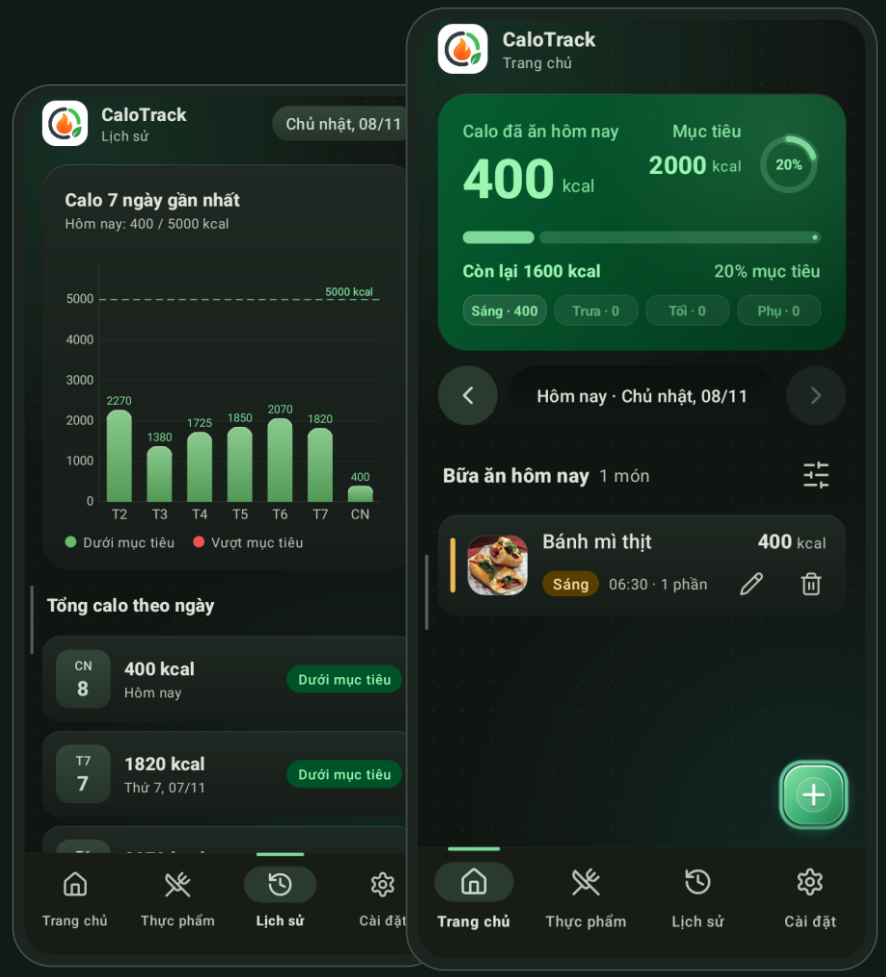
FINAL PROJECT REPORT · TOPIC 10

Calorie Counter App CaloTrack

An Android application that helps users track daily calorie intake, manage meals, and compare progress against personal goals · built with Kotlin, Room (SQLite), and Material 3.



Group [GROUP NUMBER]

University / Faculty: [UNIVERSITY / FACULTY]
Instructor: [INSTRUCTOR NAME]
Team Members: [TEAM MEMBERS]

Project Overview

CaloTrack is a self-contained offline Android application: users log daily meals by meal type and portion size, the app automatically calculates calories, compares intake against daily calorie goals, and visualizes historical trends via a daily summary list and a 7-day bar chart.

89

Pre-populated Vietnamese foods with bundled photos

4

Main screens: Home, Foods, History, Settings

2

SQLite tables managed by Room ORM (foods & meals)

3

Settings persisted in SharedPreferences (goal, unit, theme)

Platform: Android 7.0 (API 24) and above · targetSdk 34 · 100% offline local storage · no account registration or network connection required.

Motivation & Problem Statement

01

Practical Daily Nutrition Tracking

Weight loss, muscle gain, and healthy eating habits all start with knowing exactly how many calories are consumed each day.

02

Tailored for Vietnamese Cuisine

Global apps often lack local dishes; CaloTrack preloads 89 popular Vietnamese foods (Pho, Bun Cha, Com Tam, Banh Mi) with familiar serving units.

03

Comprehensive Android Engineering

Covers the core Android stack: Activity/Fragment lifecycle, RecyclerView, MVVM with LiveData/Flow, Room SQLite, SharedPreferences, and custom Canvas charts.

04

Private, Instant & Offline-First

All meal records reside securely inside the app sandbox (/data/data/com.ndtien.calotrack), launching instantly without login walls.

Objectives & Project Scope

Objectives

- Build a responsive, stable Android app supporting Android 7.0+ (API 24 to 34)
- Fulfill all 9 mandatory functional requirements specified in Topic 10
- Apply a clean layered architecture: UI (View) · ViewModel · Room DAO (SQLite)
- Persist structured records in SQLite and user preferences in SharedPreferences
- Deliver a polished Material 3 Soft-Tech UI with seamless Light & Dark modes

Project Scope

- **In Scope:** Food catalog CRUD, daily meal logging, auto calorie math, status colors, 7-day chart, settings
- **Extended Features:** 89 real food thumbnails, multi-criteria filtering, swipe-to-delete with Undo, past-day navigation
- **Out of Scope:** Cloud synchronization, barcode scanning, wearable integration, macro breakdown (Protein/Carb/Fat)

Functional Requirements (Topic 10)

01 Pre-seeded Food Database

89 basic foods initialized in SQLite on first launch; full Add, Edit, Delete support

02 Add Daily Meals

Breakfast / Lunch / Dinner / Snack; pick from catalog or Quick Entry; selectable date

03 Auto Calorie Calculation

Total = food calories x portion (0.5 · 1.0 · 1.5…); edit or delete logged meals

04 Chronological Meal List

Ordered by meal time; each card displays food photo, name, portion, and calories

05 Daily Overview & Status Color

Displays Eaten, Goal, and Remaining/Over; turns Green when under goal, Red when exceeded

06 Calorie History & Bar Chart

Daily calorie history list + 7-day rounded gradient bar chart with target limit line

07 Daily Calorie Goal

User-configurable target (500 to 10,000 kcal/day) persisted via SharedPreferences

08 Food Search & Filtering

Real-time accent-insensitive search combined with category and calorie range filters

09 Local Persistence

SQLite (Room) for foods & meals; SharedPreferences for calorie goal, unit, and theme

Target Users & Core Workflow

01 · LOG MEALS

Daily Intake

- Pick food from 89-item catalog
- Use Quick Entry for custom dishes
- Adjust portion in 0.5 steps
- Edit, swipe-to-delete, or Undo

02 · MONITOR

Real-Time Feedback

- View Eaten vs. Target calories
- 4-meal breakdown pills (B/L/D/S)
- Green status under target, Red over
- Compare 7-day bar chart trends

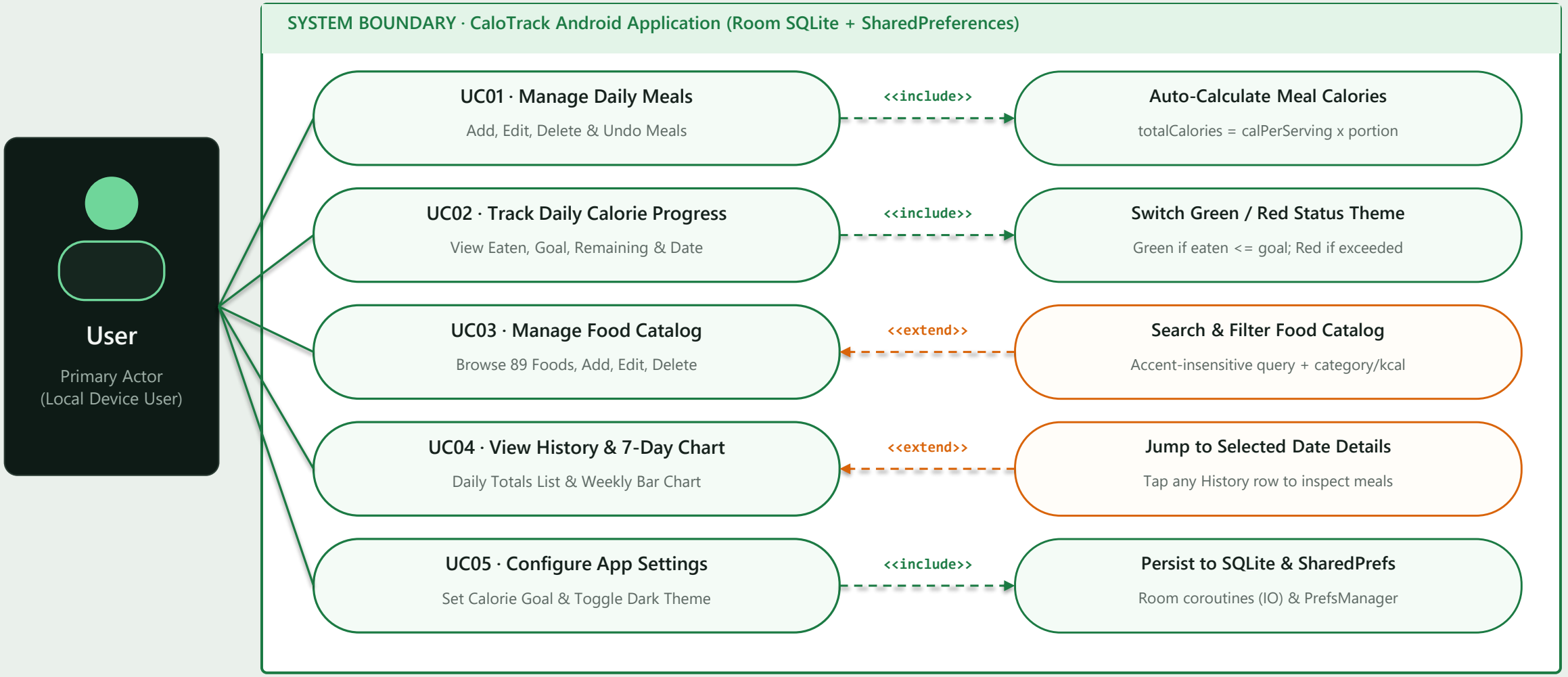
03 · CUSTOMIZE

Personal Preferences

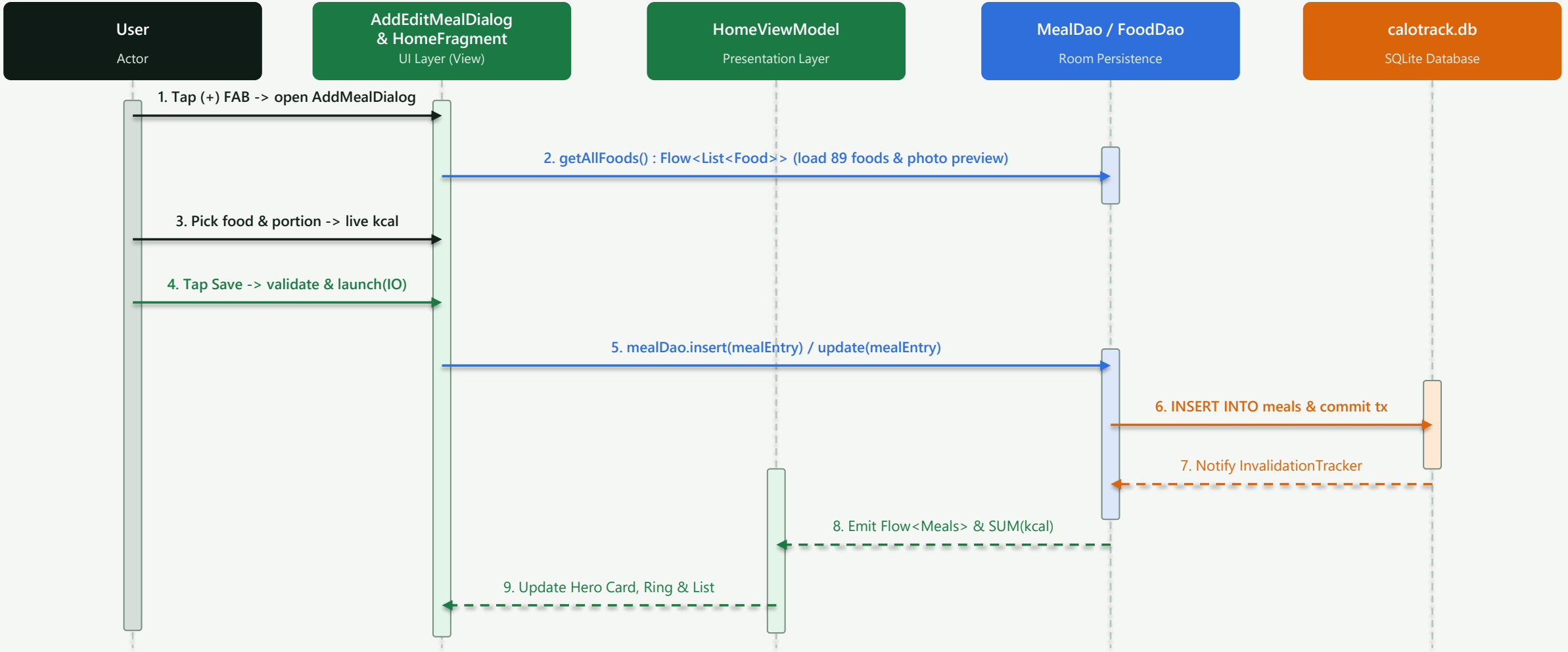
- Set daily goal (500 - 10,000 kcal)
- Add, edit, or delete custom foods
- Search & filter food catalog
- Switch Light / Dark theme smoothly

Target Audience: Individual users (students, fitness enthusiasts, dieters) seeking a fast, single-user nutrition tracker. All data is stored privately inside /data/data/com.ndtien.calotrack/.

UML Use Case Diagram



UML Sequence Diagram: Add Meal & Reactive UI Update



Overall Layered MVVM Architecture

UI Layer (View)

MainActivity + 4 Fragments, BottomSheets, Uiverse Custom Views, ViewBinding

Events / Observe →

ViewModel Layer

HomeViewModel & HistoryViewModel · LiveData, switchMap, MediatorLiveData

Queries / Flow →

Data Layer (Room SQLite)

FoodDao & MealDao returning reactive Kotlin Flow · calotrack.db

SharedPreferences

PrefsManager: target_calories, dark_mode, unit

WRITE PATH (COROUTINES)

User taps Save -> Dialog validates fields -> lifecycleScope.launch(Dispatchers.IO) executes DAO insert / update / delete off the main thread.

READ PATH (REACTIVE FLOW)

DAO returns Flow<T> -> converted via asLiveData() -> Fragment observes changes; any SQLite mutation updates the UI automatically.

Technology Stack

Layer	Technology	Role in CaloTrack
Language	Kotlin 1.9.22 · JDK 17	100% Kotlin codebase; Coroutines & Flow for asynchronous I/O
Build System	Gradle 8.6 · AGP 8.3.2 · KSP	Project compilation and compile-time Room DAO code generation
Android SDK	compileSdk 34 · minSdk 24	Supports Android 7.0 (Nougat) through Android 14
UI Framework	Material 3 (1.12.0) · ViewBinding	Type-safe view binding, BottomNavigation, Cards, Chips, BottomSheets
Custom Graphics	Android Canvas API · LruCache	UiverseCalorieRingView, UiverseOrbFabView (3D FAB), RoundedGradientBarRenderer
Architecture	AndroidX Lifecycle 2.8.0 (MVVM)	ViewModel, LiveData, switchMap, and MediatorLiveData state management
Database	Room 2.6.1 over SQLite	2 relational tables (foods, meals), compile-time verified SQL queries
Preferences	SharedPreferences (calotrack_prefs)	Persists daily calorie goal (500-10,000 kcal), energy unit, and Dark Mode
Charting	MPAndroidChart v3.1.0 (JitPack)	7-day calorie bar chart with custom 3D rounded gradient renderer

Source Code Structure

```
app/src/main/
├─ assets/foods/                89 real food photos (.jpg)
├─ java/com/ndtien/calotrack/
│  ├─ MainActivity.kt           BottomNav + Fragment host
│  ├─ UiverseCalorieRingView.kt Radial % gauge on Hero card
│  ├─ UiverseOrbFabView.kt      3D Emerald FAB + Sonar halo
│  ├─ data/                     Food · MealEntry · DAOs · DB
│  ├─ ui/home/                  HomeFragment · HomeViewModel
│  ├─ ui/meal/                  AddEditMealDialog
│  ├─ ui/food/                  FoodList · Adapter · Filter
│  ├─ ui/history/               HistoryFragment · BarRenderer
│  ├─ ui/settings/              SettingsFragment
│  └─ util/                     DateUtils · Prefs · ImageLoader
└─ res/                         Layouts · Drawables · Night Theme
```

data · Room Persistence

Food and MealEntry @Entity classes; FoodDao and MealDao SQL interfaces; SeedData with 89 dishes.

ui · Feature Packages

Modularized by screen: each package contains its Fragment, ListAdapter (DiffUtil), and ViewModel/Dialog.

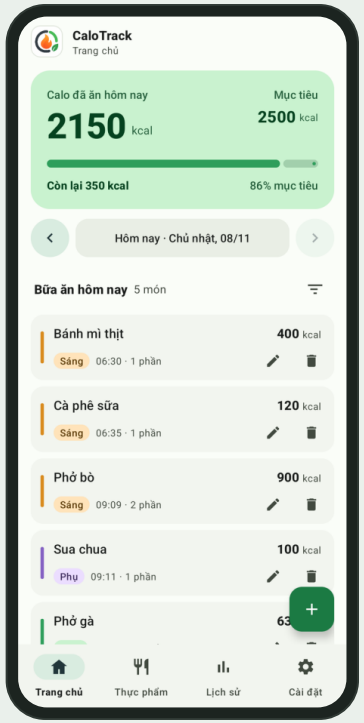
Custom Canvas Views

UiverseCalorieRingView, UiverseOrbFabView, and RoundedGradientBarRenderer deliver 60fps 3D visuals.

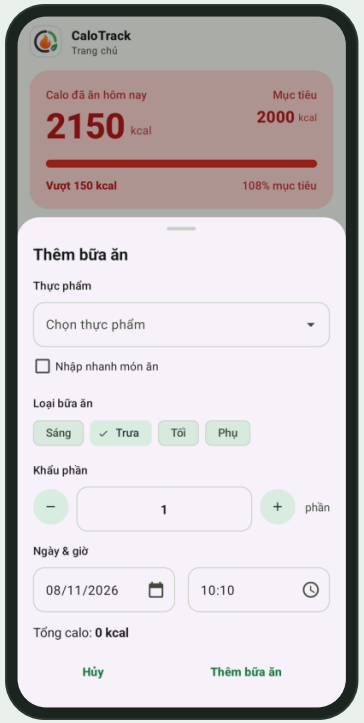
assets & res · Multi-Theme UI

89 optimized 160x160 food thumbnails cached in LruCache; full Light and Dark (values-night) themes.

User Interface Gallery (Light & Dark Modes)



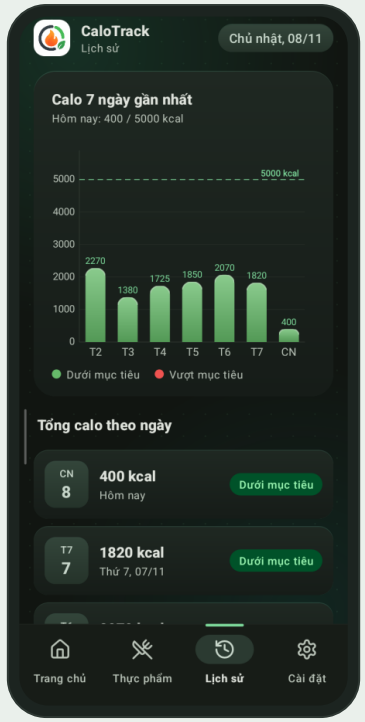
Home (Light)



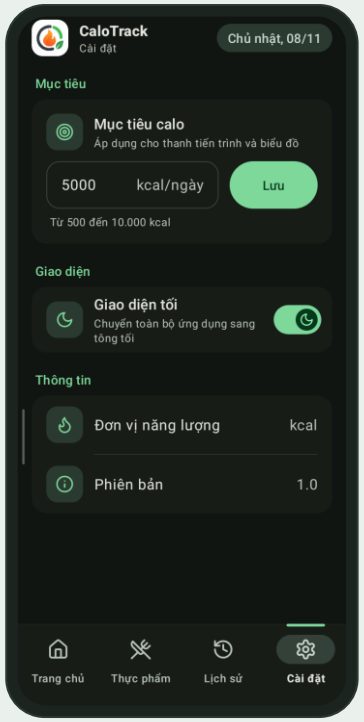
Add Meal Sheet



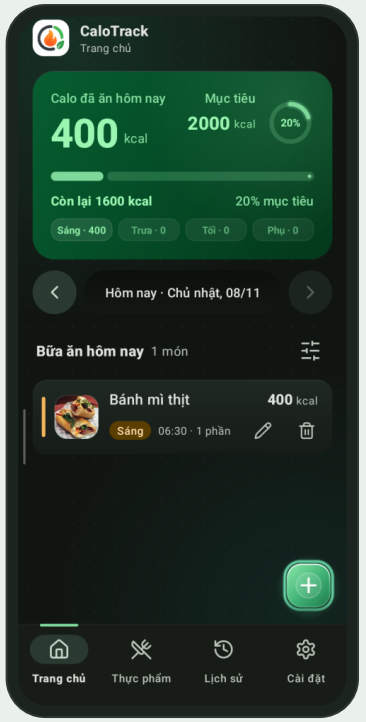
Food Catalog (89 Photos)



7-Day History Chart



Settings



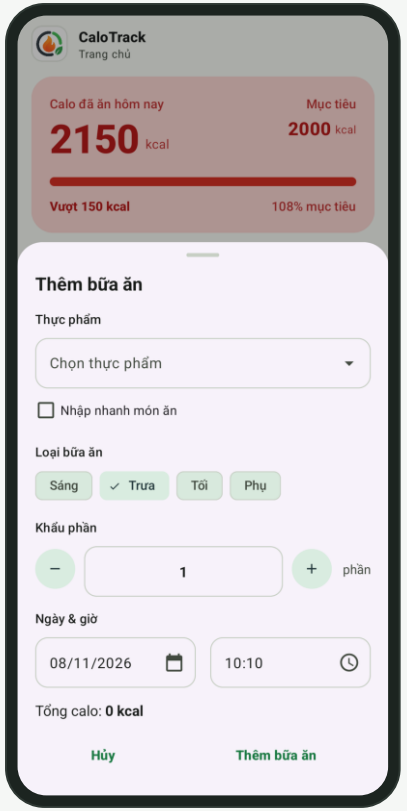
Home (Dark 3D UI)

Feature Implementation: Adding & Calculating Meals

1. Auto-select meal type by current hour (Breakfast, Lunch, Dinner, Snack)
2. Pick dish via AutoComplete dropdown (with live photo) or Quick Entry
3. Adjust portion size with - / + stepper (0.5 step; supports comma decimals)
4. Live formula preview: totalCalories = caloriesPerServing x portion
5. Validate fields -> run DAO insert/update on Dispatchers.IO -> show Snackbar

ui/meal/AddEditMealDialog.kt · saveMeal()

```
val quick = binding.cbQuickEntry.isChecked
val name = if (quick) binding.etQuickName.text
        ?.toString()?.trim().orEmpty()
    else selectedFood?.name.orEmpty()
if (name.isEmpty()) { /* show field error */ return }
val portion = parsePortion()   // "1,5" -> 1.5
if (portion <= 0) { setPortionInvalid(true); return }
val entry = MealEntry(
    id = old?.id ?: 0L,
    foodId = if (quick) null else selectedFood?.id,
    foodName = name,
    caloriesPerServing = calPerServing,
    portion = portion,
    totalCalories = calPerServing * portion,
    mealType = currentMealType(),
    date = selectedDate, time = selectedTime, ...)
lifecycleScope.launch(Dispatchers.IO) {
    if (old != null) mealDao.update(entry)
    else mealDao.insert(entry)
}
```

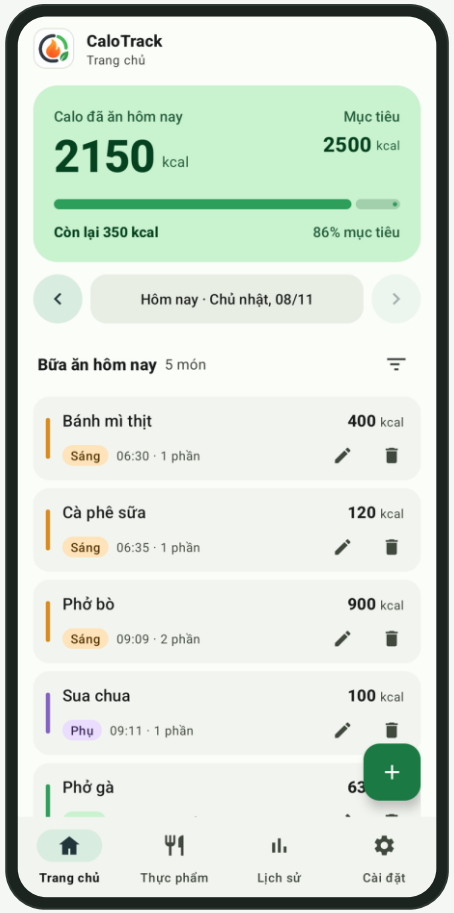


Daily Calorie Overview & Dynamic Status Color

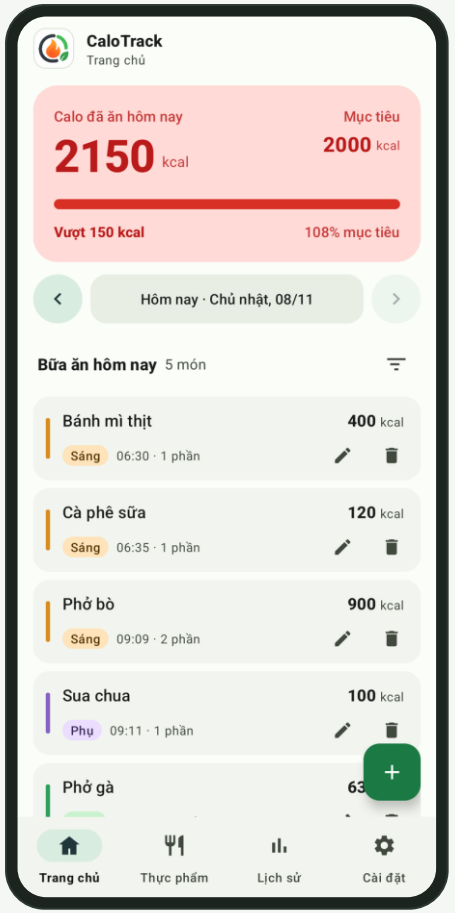
ui/home/HomeFragment.kt · updateOverview()

```
val safeTarget = target.coerceAtLeast(1)
val diff = safeTarget - eaten
val isOver = diff < 0
val pct = ((eaten * 100.0) / safeTarget).roundToInt()
tvRemain.text = if (!isOver)
    "Con lai ${fmtCal(diff)} kcal" // Remaining
else "Vuot ${fmtCal(-diff)} kcal"   // Exceeded
binding.ringCalorieGauge.setProgress(pct, isOver)

// Switch Hero card surface, text & progress bar colors:
// Green (bg_hero_under) when under goal, Red (bg_hero_over) when over
binding.layoutHeroBg.setBackgroundResource(
    if (isOver) R.drawable.bg_hero_over
    else R.drawable.bg_hero_under
)
progressCal.setIndicatorColor(barColor)
progressCal.setProgressCompat(pct.coerceIn(0, 100), true)
```



Goal 2,500 kcal -> Green



Goal 2,000 kcal -> Red

Calorie History & 7-Day Bar Chart

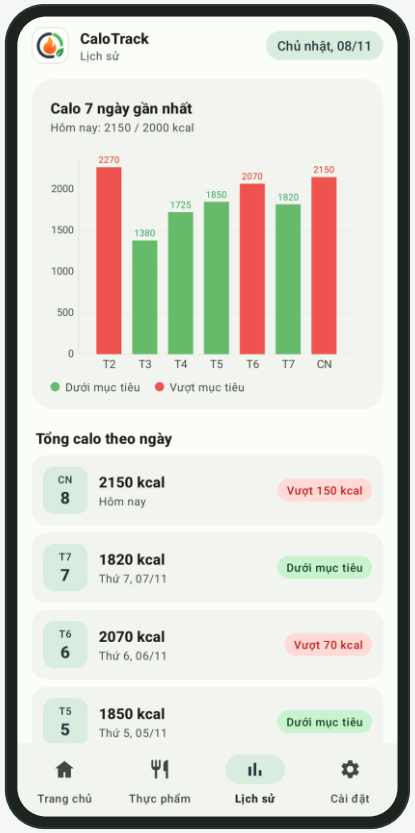
data/MealDao.kt + ui/history/HistoryViewModel.kt

```
@Query("SELECT date AS date,
  COALESCE(SUM(totalCalories),0) AS total
  FROM meals WHERE date BETWEEN :from AND :to
  GROUP BY date ORDER BY date ASC")
fun getDailyTotals(from: String, to: String)
    : Flow<List<DayTotal>>

val last7 = DateUtils.last7Days()
val weeklyTotals = mealDao.getDailyTotals(
    last7.first(), last7.last()).asLiveData()
```

ui/history/HistoryFragment.kt · updateChart()

```
val entries = days.mapIndexed { i, date ->
  BarEntry(i.toFloat(), (totalByDate[date] ?: 0.0).toFloat()) }
val overFlags = days.map { (totalByDate[it] ?: 0.0) > target }
dataSet.colors = overFlags.map { if (it) colorOver else colorUnder }
```



7-Day SQL Aggregation vs. Chart Output

Date	SUM(totalCalories)	Status
Mon · 02/11	2,270 kcal	Over Goal
Tue · 03/11	1,380 kcal	Under Goal
Wed · 04/11	1,725 kcal	Under Goal
Thu · 05/11	1,850 kcal	Under Goal
Fri · 06/11	2,070 kcal	Over Goal
Sat · 07/11	1,820 kcal	Under Goal
Sun · 08/11	2,150 kcal	Over Goal

Days without logged meals are zero-filled in memory so the chart always renders all 7 columns. Bars and value labels switch between Green and Red against the 2,000 kcal target.

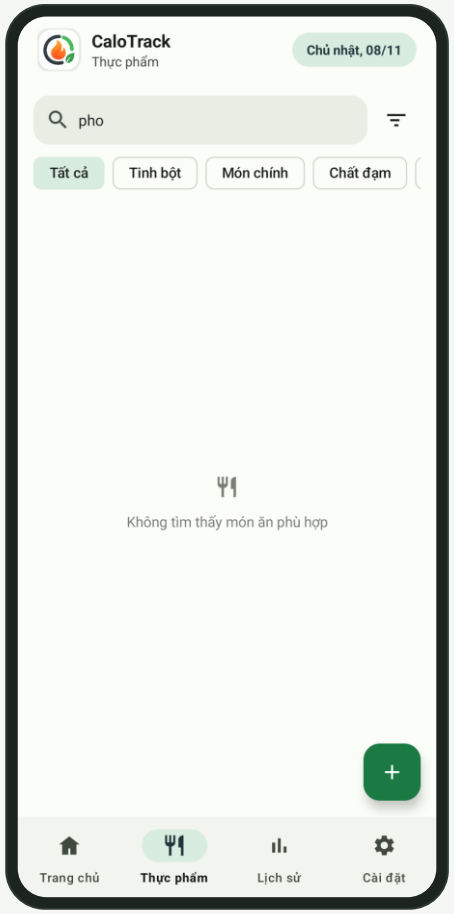
Accent-Insensitive Search, Filtering & Undo

ui/food/FoodListFragment.kt · Accent-Insensitive Search

```kotlin
private fun normalize(s: String): String {
    val nfd = Normalizer.normalize(s.lowercase(), Normalizer.Form.NFD)
    return nfd.replace(Regex("\\p{InCombiningDiacriticalMarks}+"), "")
              .replace('đ', 'd')
}
// Typing "pho" matches "Phở bò", "Phở gà", "Phở cuốn"
if (q.isNotEmpty()) {
    list = list.filter { normalize(it.name).contains(q) }
}
```

ui/food/FoodListFragment.kt · Delete with Snackbar Undo

```kotlin
private fun deleteFoodWithUndo(food: Food) {
    viewLifecycleOwner.lifecycleScope.launch {
        withContext(Dispatchers.IO) { foodDao.delete(food) }
        Snackbar.make(binding.root, R.string.food_deleted, LENGTH_LONG)
            .setAction(R.string.undo) {
                lifecycleScope.launch(Dispatchers.IO) { foodDao.insert(food) }
            }.show()
    }
}
```



Query 'pho' -> 3 Pho dishes



Delete + One-Tap Undo

Local Storage Overview: SQLite & SharedPreferences

Table: foods (SQLite)

```
id (INTEGER PK AUTOINCREMENT)
name (TEXT NOT NULL)
caloriesPerServing (REAL)
servingUnit · category (TEXT)
```

89 pre-seeded dishes across 8 categories

Table: meals (SQLite)

```
id (INTEGER PK AUTOINCREMENT)
foodId (INTEGER NULLABLE)
foodName · caloriesPerServing
portion · totalCalories · mealType
date (yyyy-MM-dd) · time (HH:mm)
```

Chronological daily meal logs

calotrack_prefs.xml

```
target_calories (Int, default 2000)
dark_mode (Boolean, default false)
unit (String, default "kcal")
demo_seeded (Boolean)
```

Lightweight key-value SharedPreferences

Room ORM Schema Generation · Compile-time KSP generates SQLite DDL from @Entity classes with auto-increment primary keys.

Snapshot Columns in meals · meals stores foodName and caloriesPerServing directly so editing/deleting a food never alters past history.

ISO-8601 Date & 24h Time Strings · Storing date as yyyy-MM-dd and time as HH:mm enables direct lexicographical sorting and BETWEEN queries.

Synchronous First-Launch Seeding · 89 Vietnamese foods are inserted inside RoomDatabase.Callback.onCreate within the initial transaction.

Entity-Relationship Diagram (ERD: 2 SQLite Tables)

foods

@Entity(tableName = "foods") · 89 Pre-seeded Rows

PK	id	INTEGER	AUTOINCREMENT, NOT NULL
	name	TEXT	Dish name (NOT NULL)
	caloriesPerServing	REAL	kcal per 1 serving
	servingUnit	TEXT	Default: "phần"
	category	TEXT	8 food groups

1 ---- 0..N

foods.id -> meals.foodId
(Logical Reference)

meals

@Entity(tableName = "meals") · Daily Meal Logs

PK	id	INTEGER	AUTOINCREMENT, NOT NULL
FK	foodId	INTEGER	NULLABLE (NULL if Quick Entry)
	foodName	TEXT	Snapshot of dish name
	caloriesPerServing	REAL	Snapshot of kcal/serving
	portion	REAL	Portion multiplier (0.5, 1.0...)
	totalCalories	REAL	caloriesPerServing * portion
	mealType	TEXT	BREAKFAST / LUNCH / DINNER / SNACK
	date	TEXT	Format: yyyy-MM-dd
	time	TEXT	Format: HH:mm
	createdAt	INTEGER	Epoch timestamp (ms)

Why a Decoupled 2-Table SQLite Design?

- **Lightweight Local Storage:** CaloTrack runs 100% on-device using SQLite, requiring only 2 core tables (**foods** and **meals**).
- **Historical Integrity: meals** copies **foodName** and **caloriesPerServing** at log time so editing or deleting a food never corrupts past history.
- **Flexible Quick Entry:** When logging an uncataloged dish via Quick Entry, **foodId** is stored as **NULL**.

SQL Queries in Room DAOs

SELECT · Daily Meals & Total Sum · MealDao.kt

```
SELECT * FROM meals
WHERE date = :date ORDER BY time ASC

SELECT COALESCE(SUM(totalCalories), 0)
FROM meals WHERE date = :date
```

GROUP BY · Historical Daily Totals · MealDao.kt

```
SELECT date AS date,
  COALESCE(SUM(totalCalories), 0) AS total
FROM meals
GROUP BY date ORDER BY date DESC
```

SELECT · Catalog & Keyword Search · FoodDao.kt

```
SELECT * FROM foods                -- getAllFoods()
ORDER BY name COLLATE NOCASE ASC

SELECT * FROM foods WHERE name     -- searchFoods(q)
LIKE '%' || :q || '%'
```

INSERT · UPDATE · DELETE · Generated by Room KSP

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insert(food: Food): Long
@Update suspend fun update(meal: MealEntry)
@Delete suspend fun delete(meal: MealEntry)
```

Database Initialization & SharedPreferences

data/AppDatabase.kt · getInstance()

```
Room.databaseBuilder(appContext,
        AppDatabase::class.java, "calotrack.db")
    .addCallback(object : RoomDatabase.Callback() {
        // Seed 89 foods synchronously inside creation tx
        override fun onCreate(db: SupportSQLiteDatabase) {
            super.onCreate(db)
            SeedData.getFoods().forEach { food ->
                db.insert("foods", CONFLICT_NONE,
                    ContentValues().apply {
                        put("name", food.name)
                        put("caloriesPerServing", ...)
                        put("servingUnit", food.servingUnit)
                        put("category", food.category)
                    })
            }
        }
    })
    .build()
```

util/PrefsManager.kt

```
private val prefs = context.getSharedPreferences(
    "calotrack_prefs", Context.MODE_PRIVATE)
var targetCalories: Int
    get() = prefs.getInt(KEY_TARGET_CALORIES, 2000)
    set(v) = prefs.edit().putInt(KEY_TARGET_CALORIES, v).apply()
var darkMode: Boolean   // default false
val unit: String get() = prefs.getString(KEY_UNIT, "kcal")...
```

ui/settings/SettingsFragment.kt

```
val value = etTarget.text?.toString()?.trim()?.toIntOrNull()
if (value != null && value in 500..10000) {
    prefs.targetCalories = value
    homeViewModel.refreshTarget() // syncs Home & History
} else { /* Toast "Please enter 500 to 10,000 kcal" */ }
```

Testing Methodology & Verification Cases

1 · Build & KSP
gradlew assembleDebug

2 · Physical Phones
Redmi 10C & Note 13 Pro 5G

3 · Emulator
Clean install & seed test

4 · DB Inspection
Verify calotrack.db & prefs via ADB

Test Case	Expected Result	Actual Result
First app launch after clean install	foods table contains 89 dishes	PASS · 89 dishes across 8 categories
Add 'Pho bo' (450 kcal), portion 1.5	Auto-calculates 675 kcal	PASS · 675 kcal · daily total 520 -> 1,195
Edit 'Pho bo' meal: portion 1.5 -> 2.0	Updates same row ID to 900 kcal	PASS · id=78 updated to 900 kcal
Quick Entry 'Yogurt' 100 kcal (Snack)	Inserts meal with foodId = NULL	PASS · foodId=NULL · 100 kcal
Tap Save with empty food selection	Shows validation error, blocks insert	PASS · Inline error shown · 0 rows added
Daily total 2,150 kcal > Goal 2,000 kcal	Hero card turns Red, shows Exceeded	PASS · Red theme · 'Exceeded by 150 kcal' (108%)
Set calorie goal to 300, then 2,500	300 rejected; 2,500 saved to prefs	PASS · Warning Toast · target_calories = 2500
Search 'pho' without accents in Foods	Matches Vietnamese 'Phở' dishes	PASS · 3 dishes: Pho bo, Pho cuon, Pho ga
Delete food item then tap Undo	Restores exact record and ID	PASS · 89 -> 88 -> 89 rows (id preserved)
Compare 7-day chart bars vs SQL SUM	Matches daily totals for all 7 days	PASS · 7/7 days match SQLite GROUP BY
Enable Dark Mode and restart app	Persists dark_mode = true	PASS · Launches directly in Dark Mode
Change goal in Settings -> switch to Home	Home & History update immediately	PASS · Updates to 2,500 kcal (86%) instantly

Key Achievements & Deliverables

9 / 9

Mandatory Topic 10 requirements completed (100%)

12 / 12

End-to-end test cases passed on real hardware

89 Photos

Real food thumbnails bundled with LruCache memory caching

7.3 MB

Self-contained debug APK with 3D Soft-Tech custom views



Repository: github.com/ndtien0406-design/app_calo · Tested on Xiaomi Redmi 10C (720x1650) and Xiaomi Redmi Note 13 Pro 5G (1220x2712 @ 480dpi).

CaloTrack · Topic 10 · Right: Dark Mode (values-night) on History and Food Catalog screens

Challenges & Engineering Solutions

Empty food catalog race condition on first launch	Seeded all 89 foods synchronously inside RoomDatabase.Callback.onCreate within the initial SQLite creation transaction
Fragment hide/show tab navigation bypassing onResume	Shared HomeViewModel via activityViewModels() and hooked onHiddenChanged() so goal updates reflect instantaneously
Accent-insensitive Vietnamese search & midnight rollover	Normalized strings with Unicode NFD diacritic stripping; added automatic date rollover check when returning to Home
Preserving historical meal accuracy when foods change	Stored snapshot columns (foodName, caloriesPerServing) in meals and made foodId nullable for Quick Entry
FAB overlapping bottom card actions while scrolling	Built UiverseOrbFabView with Scroll-Aware Auto-Dodge, 112dp list padding, Sonar pulse halo, and paired 0-135 deg icon spin

Limitations & Future Enhancements

Current Limitations

- Energy unit is currently fixed to kcal (read-only in Settings) without live kJ conversion
- Tracks total energy (calories) only, without splitting macronutrients (Protein, Carbs, Fat)
- All data resides locally on the device without JSON/CSV export or cloud backup
- Room database is at schema version 1 without automated Migration scripts for future schema changes

Future Enhancements

- Add interactive kcal / kJ unit conversion (1 kcal = 4.184 kJ) leveraging the unit key in PrefsManager
- Extend foods and meals schemas to track Protein, Carbs, Fat, and daily water intake
- Support local JSON / Excel backup export and optional Google Drive synchronization
- Add automated Room in-memory DAO unit tests and Espresso UI instrumentation suites

06 · CONCLUSION

Conclusion & Q&A

CaloTrack is a complete, production-ready Android application: fulfilling all 9 requirements of Topic 10, powered by a clean 2-table Room SQLite schema, SharedPreferences, and a 3D Material 3 interface.

Engineering

Kotlin, Room SQLite, MVVM LiveData/Flow, Coroutines, and custom 60fps Canvas views

Product

89 Vietnamese foods with photos, auto calorie math, 7-day chart, Light/Dark themes

Key Takeaways

Decoupled historical meal snapshots, reactive UI streams, real-device verification



Thank You · Questions & Answers

Thank you for your time and attention!

Group [GROUP NUMBER] · CaloTrack (Topic 10) · github.com/ndtien0406-design/app_calo